

Lab 5.3 MTConnect Client Application

3.1 '?' query and XPath example

In the previous lab, we learned the basic usage of HTTP request (data collection method). By using '?' query and XPath, you can request specific data in many senses. Some examples of '?' queries and XPath of MTConnect are below. Here are example HTTP requests but you need to change the IP address and/or port number.

- ?from=
 - It specifies the starting sequence number of the agent
 - Ex) <http://127.0.0.1:5000/sample?from=1000>
 - This example requests the agent data from the sequence number 1000.
 - Default starting sequence (without '?from=') is the first sequence of the agent.
- ?count=
 - It specifies how many sequences you want to request from the agent.
 - Ex) <http://127.0.0.1:5000/sample?count=2000>
 - This example requests 2000 count of data from the agent.
 - Default amount of data of 'sample' method is 100.
- ?path=
 - It specifies XPath (XML document path).
 - The value for the XPath is the location of the information defined in the Device Information model that represents the Structural Elements.
 - MTConnect observes standard XPath.
 - Ex1) [http://127.0.0.1:5000/current?path=//DataItem\[@id="a1"\]](http://127.0.0.1:5000/current?path=//DataItem[@id=)
 - This example requests the last data of "a1" data item.
 - Because this example is 'current' request, you will get only one (the last) data.
 - Ex2) [http://127.0.0.1:5000/current?path=//DataItem\[@type="ACCELERATION"\]](http://127.0.0.1:5000/current?path=//DataItem[@type=)
 - This example requests data that each type is "ACCELERATION".
 - By using this, you can request only acceleration data.
 - The last value of each acceleration will be responded to.
- Merge '?' queries using '&'.
 - It is possible to merge '?' queries using '&' between queries, the example is below.
 - Ex) [http://127.0.0.1:5000/sample?from=123&count=234&path=//DataItem\[@id="a1"\]](http://127.0.0.1:5000/sample?from=123&count=234&path=//DataItem[@id=)

Task 3.1

1. Request temperature and humidity data (from the sequence number 100 and count 20 amount) to MTConnect agent
 - Make sure that MTConnect agent and all MTConnect adapters are still running.
 - Hint:
 - You need to use "OR" operator because each element cannot have both temperature and humidity.
 - "OR" operator of XPath is "|".
 2. Capture the responded web browser as Figure 12 as and attach it to the report below:
-

MTConnect Device S...MTConnect Device S...view-source:192.168...rename the current t...Or operator xpath h...How to combine ANI...+--□×

←↻⚠ Not secure 192.168.50.190:5000/sample?from=100&count=20&path=//DataItem[@id="h1"%20or%20@id="t1"] Summarize🔗☆🔒🔄🌐⋮🗯 Chat

📁🏠 Homepage - Purd...📄🎓 Purdue Portal👤👤 John Blair Hooded...💰💵 Cash & stock | Sto...📁📄 ASUS E-Service📄🎓 OneCampus | Mos...📊📈 Your Courses | Gra...>📁📄 Other favorites

- creationTime: 2026-02-24T06:28:31Z
- sender: robertclaud
- instanceId: 1771912823
- version: 1.8.0.3
- bufferSize: 131072
- nextSequence: 178
- firstSequence: 1
- lastSequence: 3915

Device: sensor; UUID: 000

Device : sensor

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T06:01:00.400026Z	HumidityRelative	ACTUAL	humd	h1	105	54.7091106998669
2026-02-24T06:01:03.215824Z	HumidityRelative	ACTUAL	humd	h1	109	66.9910051135985
2026-02-24T06:01:06.031844Z	HumidityRelative	ACTUAL	humd	h1	114	51.396672657869
2026-02-24T06:01:08.847844Z	HumidityRelative	ACTUAL	humd	h1	120	67.4249573052792
2026-02-24T06:01:11.664455Z	HumidityRelative	ACTUAL	humd	h1	126	52.4611214933943
2026-02-24T06:01:14.479835Z	HumidityRelative	ACTUAL	humd	h1	135	56.5402695515594
2026-02-24T06:01:17.295822Z	HumidityRelative	ACTUAL	humd	h1	141	61.5487050627372
2026-02-24T06:01:20.111824Z	HumidityRelative	ACTUAL	humd	h1	149	66.9457276291484
2026-02-24T06:01:22.927828Z	HumidityRelative	ACTUAL	humd	h1	156	54.5326418221545
2026-02-24T06:01:25.744163Z	HumidityRelative	ACTUAL	humd	h1	164	50.6176693401692
2026-02-24T06:01:28.560202Z	HumidityRelative	ACTUAL	humd	h1	169	54.3092608507878
2026-02-24T06:01:31.375933Z	HumidityRelative	ACTUAL	humd	h1	177	68.5646385773725
2026-02-24T06:01:00.400026Z	Temperature	ACTUAL	temp	t1	104	17.375
2026-02-24T06:01:06.031844Z	Temperature	ACTUAL	temp	t1	113	17.312
2026-02-24T06:01:08.847844Z	Temperature	ACTUAL	temp	t1	119	17.375
2026-02-24T06:01:14.479835Z	Temperature	ACTUAL	temp	t1	134	17.312
2026-02-24T06:01:22.927828Z	Temperature	ACTUAL	temp	t1	155	17.375
2026-02-24T06:01:25.744163Z	Temperature	ACTUAL	temp	t1	163	17.312
2026-02-24T06:01:28.560202Z	Temperature	ACTUAL	temp	t1	168	17.375
2026-02-24T06:01:31.375933Z	Temperature	ACTUAL	temp	t1	176	17.312

- creationTime: 2026-01-08T16:15:43Z
- sender: nobinmyeong
- instanceId: 1767888907
- version: 1.8.0.3
- bufferSize: 131072
- nextSequence: 120
- firstSequence: 1
- lastSequence: 125

Device: sensor; UUID: 001

Device : sensor

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-01-08T16:15:36.698774Z	Acceleration		Xacc	a1	102	1.059118
2026-01-08T16:15:37.701795Z	Acceleration		Xacc	a1	104	1.019892
2026-01-08T16:15:40.710348Z	Acceleration		Xacc	a1	114	1.059118
2026-01-08T16:15:41.712192Z	Acceleration		Xacc	a1	118	1.098345
2026-01-08T16:15:37.701795Z	Acceleration		Yacc	a2	105	-5.060231
2026-01-08T16:15:39.707103Z	Acceleration		Yacc	a2	110	-5.021005
2026-01-08T16:15:40.710348Z	Acceleration		Yacc	a2	115	-5.060231
2026-01-08T16:15:35.696462Z	Acceleration		Zacc	a3	100	8.002226
2026-01-08T16:15:37.701795Z	Acceleration		Zacc	a3	106	8.08068
2026-01-08T16:15:38.704119Z	Acceleration		Zacc	a3	108	8.002226
2026-01-08T16:15:39.707103Z	Acceleration		Zacc	a3	111	8.08068
2026-01-08T16:15:41.712192Z	Acceleration		Zacc	a3	119	8.002226
2026-01-08T16:15:37.897372Z	HumidityRelative	ACTUAL	humd	h1	107	66.91
2026-01-08T16:15:40.713746Z	HumidityRelative	ACTUAL	humd	h1	116	52.07
2026-01-08T16:15:35.877913Z	Wattage	ACTUAL	power	p1	101	210.3
2026-01-08T16:15:36.919273Z	Wattage	ACTUAL	power	p1	103	210.211
2026-01-08T16:15:39.019498Z	Wattage	ACTUAL	power	p1	109	210.09
2026-01-08T16:15:40.067678Z	Wattage	ACTUAL	power	p1	112	209.912
2026-01-08T16:15:40.655999Z	Wattage	ACTUAL	power	p1	113	209.581
2026-01-08T16:15:41.699627Z	Wattage	ACTUAL	power	p1	117	209.594

Figure 12 Capture example for Task 3.1

Write your HTTP request in the free-form text section below, e.g.,
<http://192.168.1.2:5000/sample?from=100&count=20&path=somthing>

`http://192.168.50.190:5000/sample?
from=100&count=20&path=//DataItem[@id=%22h1%22%20or%20id=%22t1%22]`

Task 3.2 What are the cons and pros of two MTConnect configurations (a single adapter vs. multiple adapters) to collect multiple sensors' data? {display-mode:"form"}

The pros and cons of two MTConnect configurations are as follows: Single Adapter:

- Pros:
 - Easy networking setup (Only have to keep track of a single host and port)
- Cons:
 - _ Difficult to manage the entire adapter.
 - _ Multiple sensors need to be routed through the adapter.
 - * Cannot modify the sampling time per sensor
- Pros:
 - Easier to manage different devices
 - Can customize any number of adapters per sensing device
 - Can modify the sampling time per sensor
- Cons:
 - More difficult network setup (Need to keep track of multiple hosts and ports)
 - More difficult to debug when several agents are running at once

Task 3.3

✂If you are not familiar with Python programming and XML parsing, do this Task at home.

Perform Python XML parsing described below by loading the XML document ('Lab5_TASK1.3.xml') you saved in Task 1.3 and then attach the entire Python script to the report in the code block below. (Perform all in a single script.)

1. Print out 'sender', 'firstSequence', 'nextSequence', and 'lastSequence'.
 2. Print out each data element's 'name', 'timestamp', and 'value (or text)'
 3. Capture the result in console or terminal, and then attach it to the report below:
- Hint: Use [ElementTree XML API of Python](#)

Python - Python 3 (xml_parsing_example.py)

```
import xml.etree.ElementTree as ET

tree = ET.parse('xml_parsing_example.xml') # create element tree object
root = tree.getroot() # Get root element. root is the highest level of an xml tree which
includes all the elements with lower levels.

print(root[0].attrib['instanceId']) # Header is the element with index 0 under root.
root[0].attrib is a dictionary with keys of "creationTime", "sender", etc.
```

The sample code (xml_parsing_example.py) is given as above. This code shows how to print the value of 'instanceId' from 'header' of file xml_parsing_example.xml (Figure 13).

```
<?xml version="1.0" encoding="UTF-8" ?>
<MTConnectStreams xmlns:m="urn:mtconnect.org:MTConnectStreams:1.5" xmlns="urn:mtconnect.org:MTConnectStreams:1.5" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="urn:mtconnect.org:MTConnectStreams:1.5 http://schemas.mtconnect.org/schemas/MTConnectStreams_1.5.xsd">
  <Header creationTime="2024-01-04T01:47:46Z" sender="Dell-FZ" instanceId="1704332210" version="1.7.0.2" bufferSize="131072" nextSequence="9" firstSequence="1" lastSequence="8"/>
  <Streams>
    <DeviceStream name="sensor" uuid="000">
      <ComponentStream component="Device" name="sensor" componentId="s1">
        <Samples>
          <Acceleration dataItemId="a1" name="Xacc" sequence="2" timestamp="2024-01-04T01:36:50.651657Z">UNAVAILABLE</Acceleration>
          <Acceleration dataItemId="a2" name="Yacc" sequence="4" timestamp="2024-01-04T01:36:50.651679Z">UNAVAILABLE</Acceleration>
          <Acceleration dataItemId="a3" name="Zacc" sequence="7" timestamp="2024-01-04T01:36:50.651697Z">UNAVAILABLE</Acceleration>
          <Humidity dataItemId="h1" name="humd" sequence="3" subType="ACTUAL" timestamp="2024-01-04T01:36:50.651669Z">UNAVAILABLE</Humidity>
          <Temperature dataItemId="t1" name="temp" sequence="8" subType="ACTUAL" timestamp="2024-01-04T01:36:50.651701Z">UNAVAILABLE</Temperature>
        </Samples>
        <Events>
          <Availability dataItemId="avail" sequence="1" timestamp="2024-01-04T01:36:50.651508Z">UNAVAILABLE</Availability>
          <AssetChanged dataItemId="s1_asset_chg" sequence="6" timestamp="2024-01-04T01:36:50.651694Z">UNAVAILABLE</AssetChanged>
          <AssetRemoved dataItemId="s1_asset_rem" sequence="5" timestamp="2024-01-04T01:36:50.651685Z">UNAVAILABLE</AssetRemoved>
        </Events>
      </ComponentStream>
    </DeviceStream>
  </Streams>
</MTConnectStreams>
```

Figure 13: xml_parsing_example.xml used in the example to print the value of 'instanceId'

```
powershell - ME-597-IIoT - Visual Studio Code

(..venv) PS C:\Users\rd908\OneDrive - purdue.edu\Graduate School\Spring 2026\ME-597-IIoT\lab\lab5> python .\Lab5_Task3.3.py
Sender: robertclaud
First Sequence: 1
Next Sequence: 59
Last Sequence: 58
Name: Xacc
Timestamp: 2026-02-24T05:29:39.660554Z
Value: -0.3138128
Name: Yacc
Timestamp: 2026-02-24T05:29:39.660554Z
Value: -0.508399
Name: Zacc
Timestamp: 2026-02-24T05:29:39.660554Z
Value: 9.9243298
Name: humd
Timestamp: 2026-02-24T05:29:39.660554Z
Value: 51.0988358648898
Name: power
Timestamp: 2026-02-24T05:29:39.660554Z
Value: 2.72046113014221
Name: temp
Timestamp: 2026-02-24T05:29:36.758433Z
Value: 17.437
(..venv) PS C:\Users\rd908\OneDrive - purdue.edu\Graduate School\Spring 2026\ME-597-IIoT\lab\lab5>
```

```
In [ ]: # Place your code for Task 3.3 here:
#
import xml.etree.ElementTree as ET

tree = ET.parse("./exports/Lab5_TASK1.3.xml") # create element tree object
root = (
    tree.getroot()
) # Get root element. root is the highest level of an xml tree which includes all the elements with lower L

# Task 3.3_1: Print the values of "sender", "firstSequence", "nextSequence", and "lastSequence" in the Header
# Header is the element with index 0 under root. root[0].attrib is a dictionary with keys of "creationTime",
print(f"Sender: {root[0].attrib['sender']}")
print(f"First Sequence: {root[0].attrib['firstSequence']}")
print(f"Next Sequence: {root[0].attrib['nextSequence']}")
print(f"Last Sequence: {root[0].attrib['lastSequence']}")

# Task 3.3_2: Print out each data element's 'name', 'timestamp' and 'value (or text)'

samples = root[1][0][0][0]
for data_item in samples:
    print(f"Name: {data_item.attrib['name']}")
    print(f"Timestamp: {data_item.attrib['timestamp']}")
    print(f"Value: {data_item.text}")

#
```

Lab5 Summary and Deliverables

Answer the following questions for your achievements

Q1. Please summarize Lab5.

In Lab 5 we continued to explore the MTConnect protocol. First we hooked up the DS18B20 Temperature Sensor and the ADXL345 3-axis accelerometer to the raspberry pi and verified that the sensors worked. In addition, we connected to the vacuum pump power meter for additional monitoring data. Next, we created an MTConnect adapter to take in sensor inputs from these data sources and added a random number generator to simulate a humidity sensor. These values were then read through an MTConnect agent and viewed in a web browser. Next, we explored using multiple adapters. we set up separate adapters for the temperature and humidity, the accelerometer, and finally the power meter. At first these

adapters all ran off of the raspberry pi, but afterwards the power meter adapter was moved to the laptop. Finally, we explored using XPATH to query specific pieces of data from the MTConnect agent. This was done using query parameters in HTTP GET requests through the browser. We also developed a simple parser for a saved xml file.

Q2. What skills did you have to develop to accomplish this project?

To accomplish this project, I developed more networking skills, the use of ssh and tmux, and more python skills.

Q3. What aspects of this project were the most beneficial for your learning?

The aspects of this project that were the most beneficial for my learning were setting up multiple adapters individually catered to specific sensor outputs. This made the idea of adapters click in my head as I was able to customize each one for a specific use case.

Q4. What challenges did you encounter in completing the project?

The challenges I encountered in completing this project were not having access to pretty print when using my laptop, and getting the data to stream from my laptop to the raspberry pi.

Q5. How did you overcome the challenges or remedy the problems encountered?

To overcome the challenge of not having pretty print, I did some research using ChatGPT and realized that windows uses unique newline characters in it's files. This can make xsl styles difficult for browsers to read and is difficult to diagnose since newline characters are invisible. I was able to replace the windows newline characters "\r\n" with standard newlines "\n" using a VSCode command.

To overcome the challenge of not being able to send data from my laptop to the raspberry pi, I did some research into hosts and IPs and found that if I switched my host from 127.0.0.1 to 0.0.0.0 it would expose my stream to my network, allowing the MTConnect agent running on my pi to receive the data from my laptop.

Get back to [Lab Index Page](#)